Explanation Readiness — What We Did, How, and Why

Companion to `STAGE1_REPORT.md`. This file is for the oral/viva part of Stage 1 evaluation — the guide is explicit that you'll be asked to explain, in your own words, how a BVAR's Minnesota prior works, how a TSFM tokenizes a series, the RMSFE metric, and the hybrid model logic. Notes below are written to actually answer those questions, not just point at code.

---

1. What problem are we solving, end to end?

Given a macro time series up to today (say, unemployment rate transformed to a monthly change), predict its value 1, 3, 6, and 12 months ahead. Do this for 60 different series, at 48 different "as of today" dates spread over 2010–2022, for 9 different forecasting methods, then ask: *which methods beat the dumbest possible forecast (an AR(1))?*

2. The data pipeline (`prep_data.py`)

FRED-MD ships with a **transform code** per series (1–7): some series are already stationary (interest rates — code 1, no transform), some need first-differencing (code 2), most need log-differencing (code 5, ~ monthly % change), a few need double-differencing of logs (code 6). This matters because forecasting models assume the data is at least approximately stationary (constant mean/variance over time) — GDP in raw dollar levels trends upward forever and no linear model can sensibly extrapolate that; GDP's *log growth rate* fluctuates around a stable mean and is forecastable. We apply the paper's own transform codes (Appendix C), verified column-by-column against our actual CSV headers.

3. AR(1) — the yardstick

`y_t = c + phi*y_{t-1} + eps_t`, fit by OLS. Simple, but a genuinely tough benchmark for economic data because most macro series *are* close to a random walk (or at least highly persistent) after transformation — beating it consistently is a real achievement, not a low bar. Every other model's score in this project is reported as a **ratio to this benchmark's error**, not an absolute number.

4. RMSFE — the scoring metric

```
RMSFE_ijh = sqrt( Sum_tau e^2_{i,j,tau+h} / Sum_tau e^2_{AR1,j,tau+h} )
```

For model *i*, series *j*, horizon *h*: take the squared forecast errors from every one of the 48 forecast origins, sum them, do the same for AR(1)'s errors on the same origins/series/horizon, and take the square root of the ratio. **A value of 0.90 means model *i*'s errors are, in aggregate, 90% the size of AR(1)'s error — i.e. a 10% improvement.** Values are computed *per series per horizon*, then we look at the *median across all 60 series* to summarize "how does this model do overall" (median rather than mean because a few series can have huge outlier errors that would otherwise dominate a simple average).

Why relative, not absolute error (e.g. plain MSE)? Because different macro series live on wildly different scales (interest rate changes vs. GDP growth vs. stock index returns) — an "MSE of 0.02" is meaningless without knowing which series it's for. Dividing by each series' own AR(1) error puts every series on the same normalized 0-to-infinity scale where 1.0 always means "as good as the naive benchmark," making cross-series aggregation (the median) meaningful.

5. The BVAR and its Minnesota prior

A VAR(p) regresses each of the 60 series on p lags of *all 60 series* (itself and 59 others). With p=6 lags and 60 series, that's 6×60+1 = 361 regressors *per equation, per series* — vastly more parameters than a typical estimation window has

data points for. Unrestricted OLS would badly overfit.

The Minnesota prior's core idea: most macro series depend mostly on their *own* recent past, only weakly on other series, and less on *older* lags than *recent* ones. So instead of letting the data freely estimate all 361 coefficients, we push (shrink) them toward a prior belief: - Coefficient on a series' own first lag: prior mean near 1 for series in levels (random-walk-like), or near **0 for already-differenced/stationary series** (which is our case, since FRED-MD data is pre-transformed to stationarity) — meaning our default guess before seeing data is "no persistence," and the data has to convince us otherwise. - Coefficient on any lag *l*: prior variance shrinks like *1/l^2* — lag 6 is shrunk 36× harder toward zero than lag 1, formalizing "the more distant past matters less."

How it's implemented without heavy Bayesian machinery: instead of literally computing a posterior distribution, we add fake ("dummy") data rows to the regression that encode exactly this prior belief, then run ordinary least squares on {real data + dummy rows} together. A dummy row with a large coefficient on lag-1's own-column and target 0 says, in effect, "pretend you also observed evidence that this coefficient should be near zero" — and the *strength* of that fake evidence (controlled by lambda, and scaled up by the lag number for the shrinkage-by-distance effect) determines how hard the prior pulls the final estimate away from what OLS alone would say. This is mathematically equivalent to a proper Bayesian Normal-Inverse-Wishart posterior mean (Banbura et al. 2010) but implemented with one `lstsq` call instead of matrix-algebra posterior formulas.

A bug we found and fixed here: the lag-decay multiplier was accidentally always evaluating to 1 (a Python conditional expression that looked like it varied by lag but didn't), so every lag got the *same* shrinkage strength regardless of distance — silently defeating half the point of the Minnesota prior. Comparing our early results (median RMSFE ~1.3, clearly wrong given the paper's ~0.98) against the paper's reported pattern is what surfaced this.

6. The Factor model

Extract the first principal component (PC1) of all 60 (standardized) series — a single number per month that captures the dominant co-movement across the whole economy (loosely: "how strong is the business cycle right now"). Then forecast each series using its own lags *plus* lags of this common factor (paper's Eq. 6). We implement this as a **direct** regression: to forecast 12 months ahead, we don't chain together twelve 1-month-ahead forecasts (each with its own error compounding into the next); we directly regress "value 12 months from now" on "value and factor level today," fitting a separate regression per horizon. This avoids the numerical instability of iterating an estimated model forward many steps (which is exactly the bug we found and fixed in an earlier version — iterating a coefficient just slightly above 1 for 12 steps blows up exponentially).

7. TSFMs: how they actually work

A Time Series Foundational Model is architecturally a small language model where the "vocabulary" is numbers instead of words. Three ingredients turn a raw numeric sequence into something a transformer can process:

1. **Scaling**: normalize the series (e.g., subtract median, divide by inter-quartile range) so that a value of "50" in an interest-rate series and "50" in a housing-starts series look the same to the model — otherwise different units/scales would confuse a model trained across thousands of unrelated datasets.
2. **Patching** (used by Moirai, TimesFM, TTM): chop the scaled series into short fixed-length windows (e.g., 32 consecutive points), each patch becomes one "token" — analogous to how a sentence is chopped into words. This lets the model attend over patches (long-range structure) instead of over every single raw timestep (much more expensive).
3. **Quantization** (used by Chronos): instead of patches, divide the *value range* into a fixed number of bins (e.g., 4096 bins spanning the series' scaled range) and represent each timestep as "which bin did this value fall into" — literally converting numbers into a small vocabulary of discrete symbols, then applying the exact same next-token-prediction machinery as a text LLM.

Pretraining: the model sees a huge corpus of unrelated time series (energy, web traffic, weather, finance, synthetic data) and learns to predict the next patch/token given previous ones — self-supervised, no labels needed, just "predict what comes next" repeated across millions of series.

Zero-shot forecasting: at inference time, we feed our FRED-MD series (which the model has *never specifically been trained on* — or may have, in some cases, more on this below) into the pretrained model with no further training, and it autoregressively (or, for the newer "Bolt"/direct-regression variants, in one forward pass) predicts the next several values.

A caveat we inherited from the paper: several TSFMs (Moirai, TTM, LagLlama) list the Monash forecasting archive in their training data, which itself includes a large part of FRED-MD. This means the "zero-shot" evaluation may not be fully out-of-sample for those models — they may have partially memorized the answer. We didn't attempt to fix this (neither did the paper, beyond flagging it), but it's an important caveat to state if asked "is this a fair test?"

Which variant did we use for Chronos, and why does it matter? Chronos has two generations: the original (T5-style, quantization + token-by-token autoregressive sampling, which we found took ~13 minutes per forecast origin on CPU — 10+ hours for the full run) and "Bolt" (predicts all quantiles for the whole horizon in a single forward pass, no autoregressive sampling loop). We switched to Bolt after discovering the speed problem; it's Amazon's own faster successor at the same parameter scale, not a smaller/weaker substitute — the architecture change, not model size, is what buys the ~2000x speedup.

8. The hybrid TSFM-BVAR model

Motivation: if BVAR is wrong about series A and a TSFM is wrong about series B, but they're not wrong about the *same* series in the *same* way, then combining their information should help — this is why the paper computes the correlation between different models' per-series errors and finds it's low (~0.07–0.5) between BVAR and the good TSFMs at h=1.

Mechanics (3 steps): 1. Fit the BVAR as usual, get its forecast y_hat_BVAR for the target date, *and* also compute its **in-sample residuals** — for every past month in the training data, "how wrong was BVAR's own 1-step prediction back then" (actual minus BVAR's fitted value). 2. Feed *that residual series* (not the original data) into a TSFM, zero-shot, and ask it to forecast the residual forward. The idea: if BVAR's mistakes have any pattern at all (e.g., it systematically under-predicts during expansions), a flexible nonlinear model might catch that pattern even though a linear BVAR structurally can't represent it. 3. Final hybrid forecast = BVAR's forecast + the TSFM's forecast of BVAR's own future error. If the TSFM correctly anticipates that BVAR is about to be too low, the correction pushes the final number up toward the truth.

What we found: this helped modestly at short and long horizons but didn't beat standalone TimesFM, and during COVID the hybrid inherited BVAR's severe breakdown almost unchanged. The residual-correction step only fixes learnable, recurring bias in BVAR's mistakes — it can't rescue a forecast when BVAR's entire linear structure becomes wrong all at once (a genuine regime break), because at that point BVAR's "residual" isn't a repeating pattern the TSFM could have learned, it's a one-off shock.

9. Why quarterly origins instead of monthly, why 2010–2022, why 60 series

Purely computational — see `STAGE1_REPORT.md` §3 for the full table. The one-sentence version: monthly re-estimation of 6 TSFMs + 2 econometric models over 38 years on a laptop CPU (no A6000 GPU, unlike the paper) would take days; quarterly origins over a 12-year window preserve a genuine out-of-sample test (including a real regime break, COVID) at a runtime that's actually completable.